# ANÁLISIS DEL RETO

*1.Nicolás Miguel Murillo Cristancho, n.murilloc@uniandes.edu.co, 202213891*

*2. Juan Pablo Peña Jaime, j.penaj@uniandes.edu.co, 202212756*

*3. Sara Sofía Cárdenas Rodríguez, ss.cardenas@uniandes.edu.co, 202214907*

## Carga de datos

### Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Si se implementó, hecho por el grupo en conjunto. |

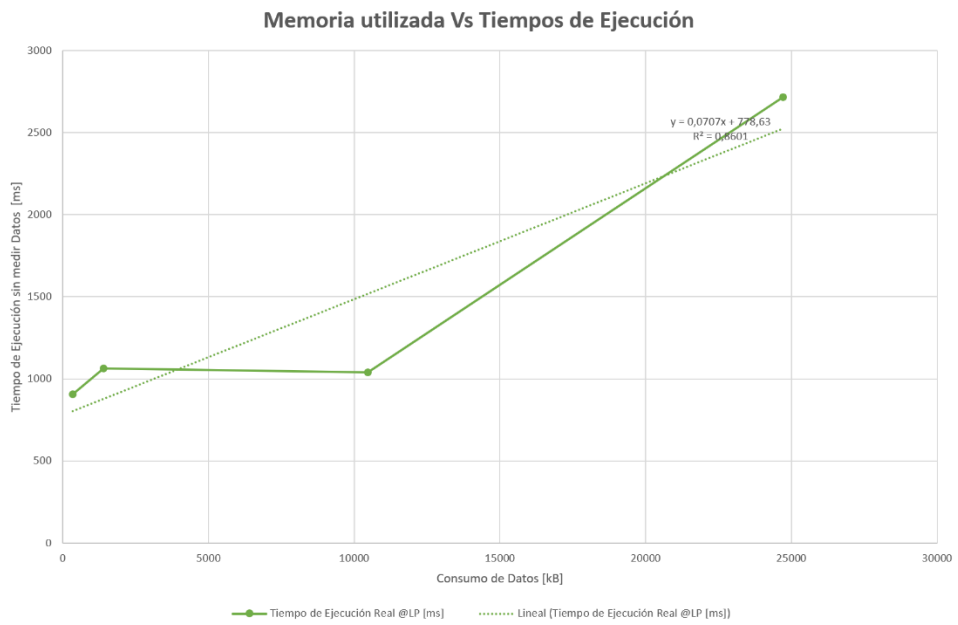
### Análisis de complejidad

| **RETO 4** | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar llaves del catálogo y archivo de lectura | O(1) |
| 2. Añadir vertices (stops) | O(V) |
| 3. Añadir arcos (edges) | O(E) |
| 4. Dar totales | O(1) |
| 5. Añadir número de adyacentes (degree) | O(V) |
| ***TOTAL*** | ***O(V+E)*** |

### Pruebas Realizadas

| **Carga** | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 2715.528 | 24706.906 |
| 50% | 1039.951 | 10464.146 |
| 20% | 1062.084 | 1406.326 |
| 5% | 906.552 | 344.460 |

### Gráficas




## Análisis

La carga tomó el tiempo y la memoria esperada según el análisis de complejidad, que depende de V y E

# Requerimiento 1

## Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Si se implementó, hecho por el grupo en conjunto. |

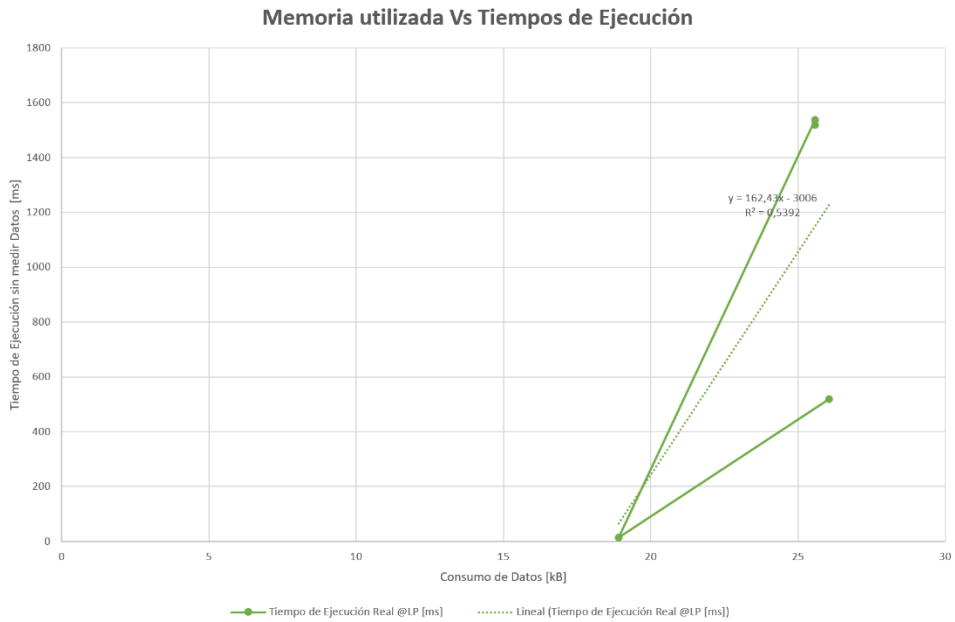
## Análisis de complejidad

| **RETO 4** | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar variables | O(1) |
| 2. Ejecutar BFS | O(V+E) |
| 3. Obtener arcos | O(N) |
| ***TOTAL*** | **O(V+E+N)** |

## Pruebas Realizadas

| **1097-55 y 357-119** | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 519.667 | 26.062 |
| 50% | 14.734 | 18.914 |
| 20% | 1538.018 | 25.578 |
| 5% | 1517.868 | 25.578 |

## Gráficas




## Análisis

La ejecución del BFS demostró que la complejidad teórica (la cual depende de V y E) se cumple en nuestro grafo

# Requerimiento 2

## Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Si se implementó, hecho por el grupo en conjunto. |

## Análisis de complejidad

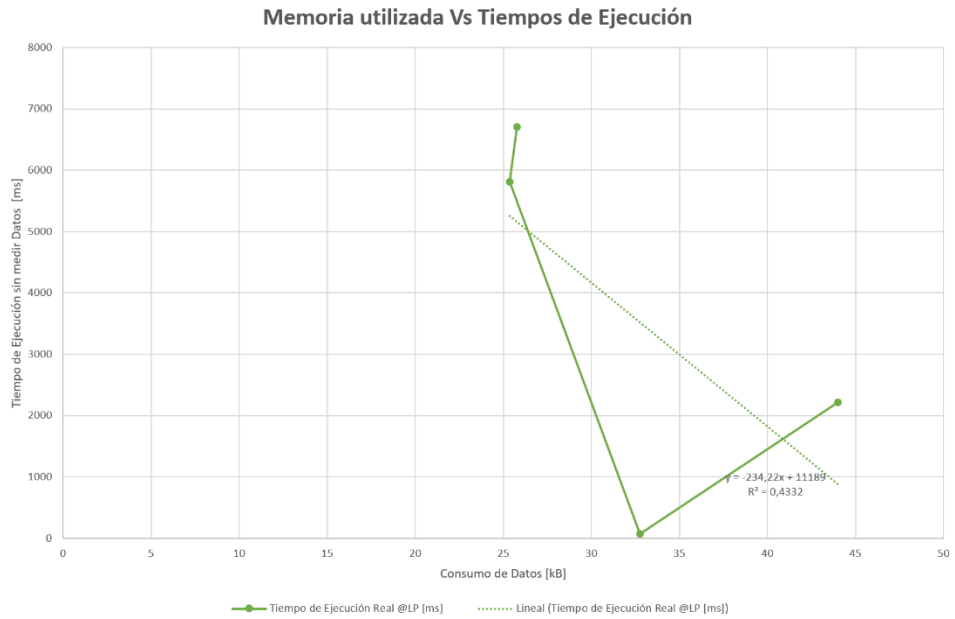
| **RETO 4** | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar variables | O(1) |
| 2. Ejecutar Dijkstra | O(E log V) |
| 3. Obtener arcos | O(N) |
| ***TOTAL*** | **O((E log V) + N)** |

## Pruebas Realizadas

| **1097-55 y 357-119** | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 2215.639 | 44 |

| | | |
|---|---|---|
| 50% | 75.053 | 32.758 |
| 20% | 5813.754 | 25.352 |
| 5% | 6700.346 | 25.773 |

## Gráficas




## Análisis

La utilización de Dijkstra nos permitió obtener el camino con menos conexiones, con una complejidad similar a la esperada según la teoría

# Requerimiento 3

## Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Sí, por Nicolás Murillo |

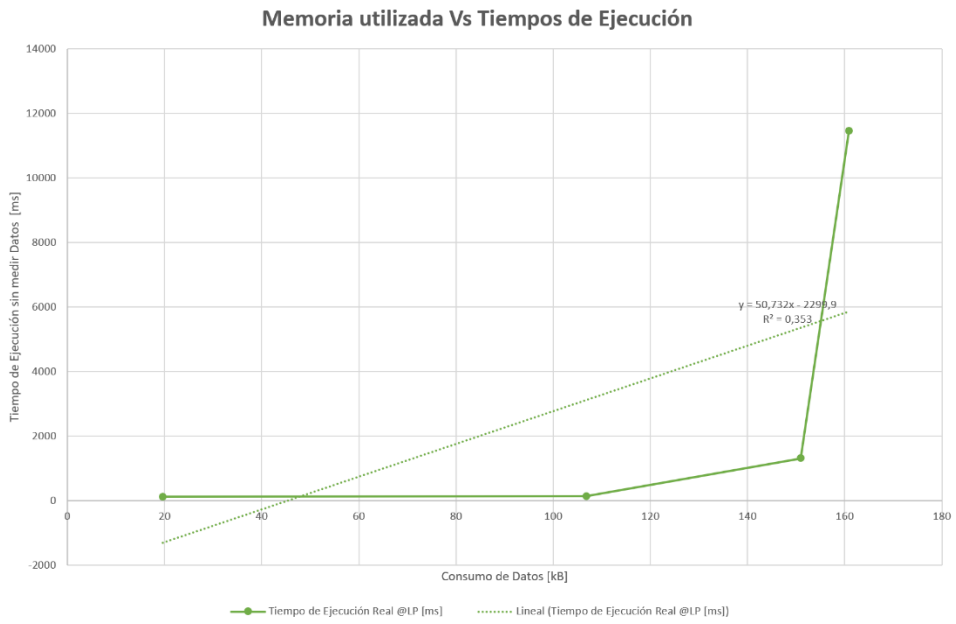
## Análisis de complejidad

| **RETO 4** | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar variables | O(1) |
| 2. Ejecutar Kosaraju | O(V+E) |
| 3. Iterar sobre estaciones para encontrar SCC común y formar lista | O(V) Vértices |
| 4. Merge sort | O(C log C) Componentes |
| ***TOTAL*** | **O(2V + E + C log C)** |

## Pruebas Realizadas

| Componentes conectados | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 11460.05 | 160,832 |
| 50% | 1315.433 | 150.919 |
| 20% | 133.457 | 106.812 |
| 5% | 122.967 | 19.648 |

## Gráficas




## Análisis

El algoritmo de kosaraju utiliza varios algoritmos anteriores, además de que se deben hacer iteraciones y ordenamientos, es por ello que este requerimiento tiene un tiempo de respuesta relativamente alto con los archivos más pesados, aun así, no se superaron los 12 segundos mediante su ejecución y tampoco se consumió memoria en exceso

# Requerimiento 4

## Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Sí, por Juan Pablo Peña |

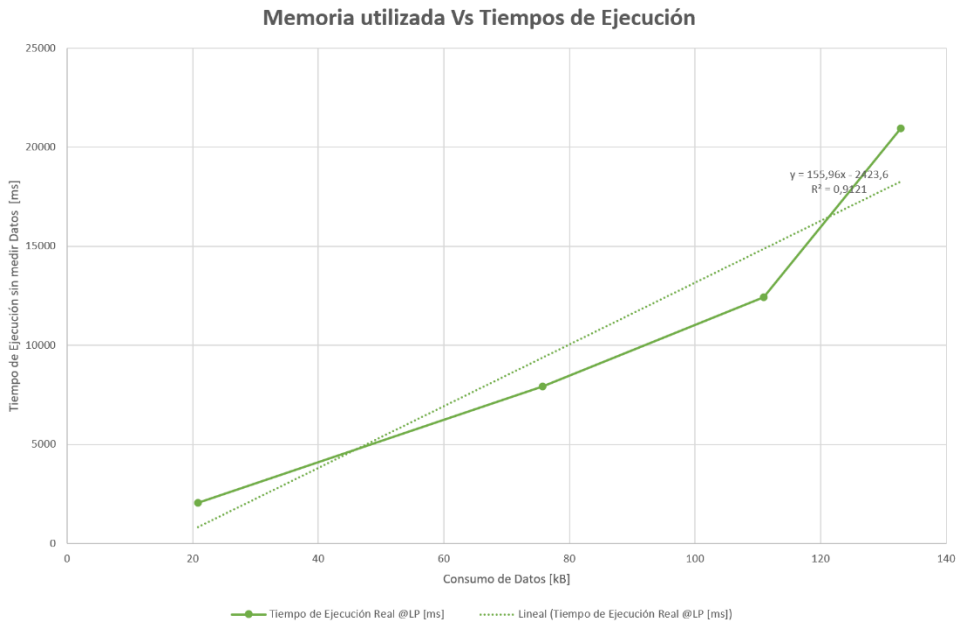
## Análisis de complejidad

| RETO 4 | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar variables | O(1) |
| 2. addPoints() | O(V) Vértices |
| 3. Ejecutar Disjktra entre Origen y Destino | O(E log V) |
| ***TOTAL*** | **O(V + E log V)** |

## Pruebas Realizadas

| **Lon: 2.138367, Lat: 41.4220**<br>**Lon: 2.148960, Lat: 41.42166** | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 20960,290 | 132.733 |
| 50% | 12437,815 | 110,932 |
| 20% | 7925,013 | 75,731 |
| 5% | 2046,236 | 20,834 |

## Gráficas




## Análisis

El algoritmo principal de este requerimiento es dijsktra, junto con el cual se ejecutan otras operaciones, debido a la complejidad de dijkstra y la memoria que usa, se obtuvieron algunos tiempos altos y también memoria. Sin embargo, se comprobó la eficacia de este algoritmo.

# Requerimiento 5

## Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Sí, por Sara Cárdenas |

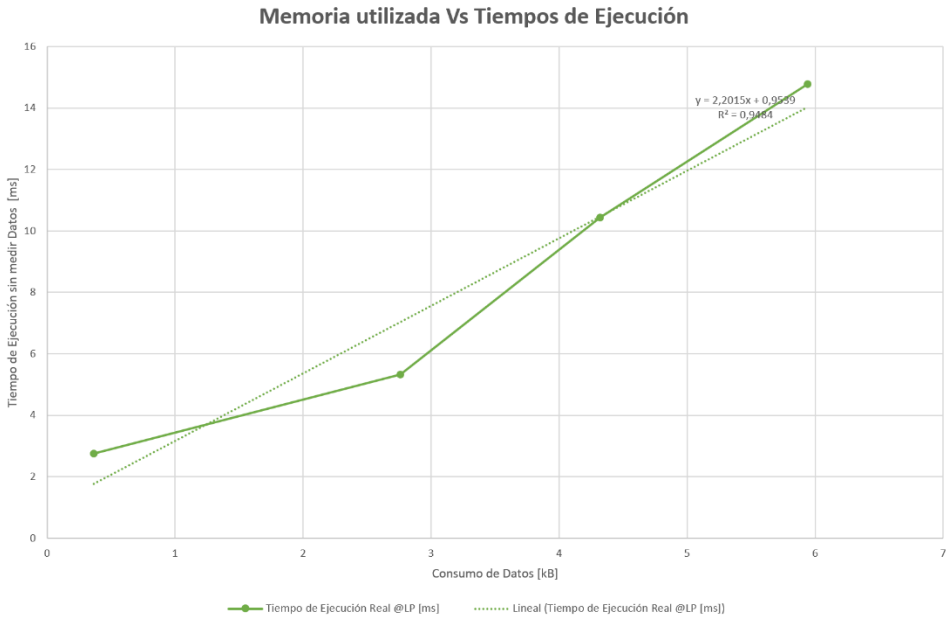
## Análisis de complejidad

| **RETO 4** | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar variables | O(1) |
| 2. addAdyacents() | O(A*C)<br>Adyacentes * Conexiones |
| 3. Obtener reporte y total | O(1) |
| ***TOTAL*** | **O(A*C)** |

## Pruebas Realizadas

| **54-V3 y 3 conexiones** | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 14,784 | 5.94 |
| 50% | 10,437 | 4,32 |
| 20% | 5,321 | 2,76 |
| 5% | 2,745 | 0,367 |

## Gráficas



## Análisis

Aquí al crear nuestra propia algorítmica entendimos la naturaleza de los vértices adyacentes y su bifurcación a través de las conexiones. Además, las pruebas demuestran que nuestro análisis de complejidad es correcto

# Requerimiento 6

## Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Si se implementó, hecho por el grupo en conjunto. |

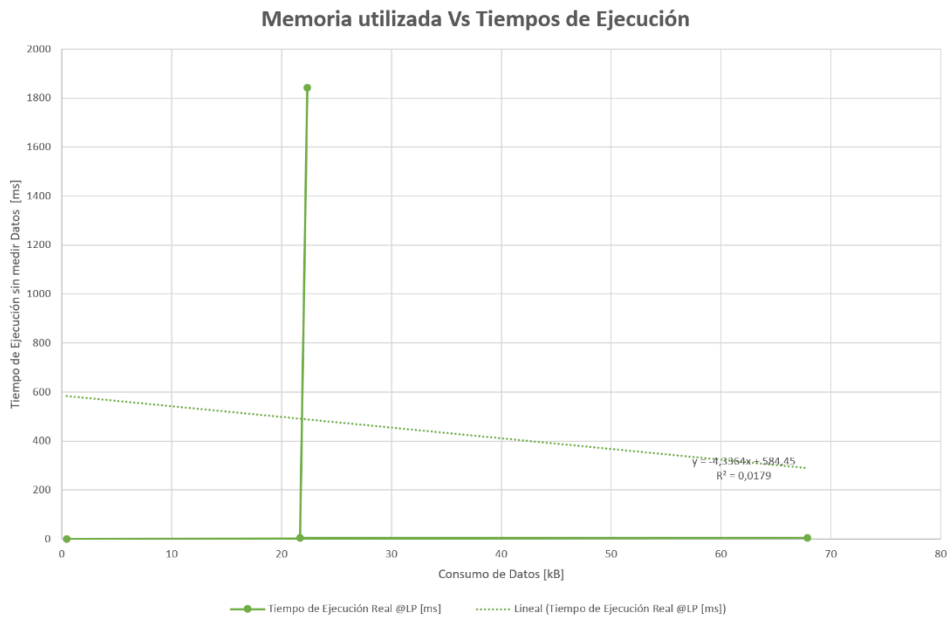
## Análisis de complejidad

| **RETO 4** | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar variables y obtener estaciones del barrio | O(1) |
| 2. Ejecutar Dijsktra | O(E log V) |
| 3. Encontrar menor distancia para obtener menor camino | O(V) Vértices del barrio |
| 4. Añadir indicativos de los barrios de origen y destino | O(N) |
| ***TOTAL*** | **O(V + E log V + N)** |

## Pruebas Realizadas

| **340-119 y la Barceloneta** | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 1842.656 | 22.324 |
| 50% | 3.776 | 21.682 |
| 20% | 3.872 | 67.873 |
| 5% | 0.361 | 0.461 |

## Gráficas




## Análisis

Se puede evidenciar que la complejidad del algoritmo depende de la cantidad de estaciones en el barrio, la cantidad de arcos y la cantidad de estaciones para llegar a dicho vecindario.

# Requerimiento 7

## Descripción

| | |
|---|---|
| **Entrada** | Archivo de datos |
| **Implementado (Sí/No)** | Si se implementó, hecho por el grupo en conjunto. |

## Análisis de complejidad

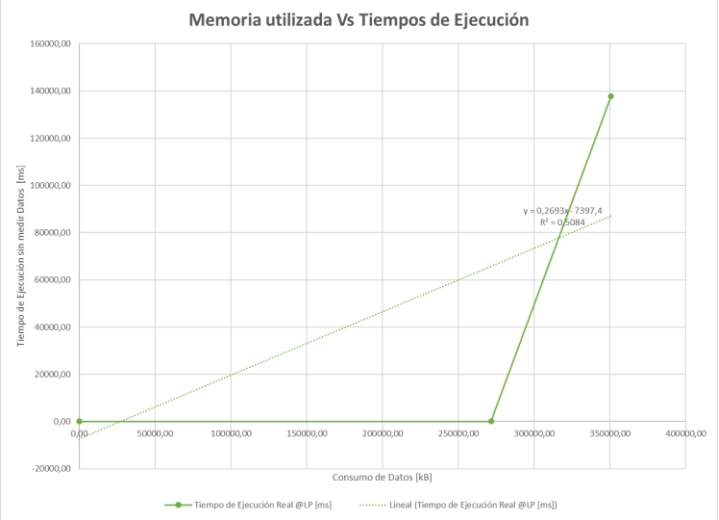
| **RETO 4** | |
|---|---|
| **Pasos** | **Complejidad** |
| 1. Inicializar variables y obtener estaciones del mismo SCC | O(1) |
| 2. Crear grafo SCC | O(V**E) V y E del SCC |
| 3. Hallar camino circular (Dos BFS y una iteración sobre V) | O(3V+2E) V y E del SCC |
| 4. Construir camino | O(2V) V y E del SCC |
| ***TOTAL*** | **O(V**E + (5V + 2E))** |

## Pruebas Realizadas

| **1663-81** | | |
|---|---|---|
| **Entrada** | **Tiempo (ms)** | **Memoria (kB)** |
| Large | 350.702 | 137.883 |
| 50% | 271.623 | 97.80 |

| | | |
|---|---|---|
| 20% | 198.52 | 50.40 |
| 5% | 50.25 | 30.25 |

## Gráficas




## Análisis

Las pruebas en este requerimiento para una misma estación son bastante fluctuantes, debido a que la cantidad de estaciones en un SCC cambia drásticamente en cada tamaño de archivo. Sin embargo, es evidente que depende de la cantidad de vértices y edges del SCC, al igual que de los resultados del BFS